

Distributed Systems — Docker & Local Kubernetes Project

Technical Report — MSc DE1 — Containerization, Security and Orchestration of the UBC Flask Sample App

Starter application	https://github.com/ubc/flask-sample-app
Repository	https://github.com/ORebecca/msc-de1-distributed-systems-docker-k8s
Docker Hub image	rebecca16ouatt/msc-de1-flask-app — tags 1.0.0 (deployed), latest, 1.1.0 (demo)
Local cluster	kind — 1 control-plane + 2 workers, namespace msc-de1-project

1. Baseline

Before any containerization work, the original, unmodified application was run locally in a Python 3.14 virtual environment to establish a working baseline:

1. Created a venv and installed `requirements.txt` (`flask==2.3.3`).
2. Ran `python -m unittest discover tests` — all 4 existing unit tests passed (`test_hello_route`, `test_add_item_route`, `test_get_item_route`, `test_get_nonexistent_item_route`).
3. Ran `python run.py` and exercised every documented route with `curl`: `GET /` (200), `GET /items` (200), `POST /items` (201), `GET /items/<id>` (200 / 404 for a missing id).

All routes behaved exactly as documented in the starter README. Raw command output is saved under `evidence/baseline/` in the repository.

2. Dockerization

The Dockerfile was written from scratch, addressing all 12 mandatory practices from the assignment brief: an official Python base image, a `WORKDIR`, dependency-first copying for layer caching, no unnecessary caches/files, a non-root runtime user, a single exposed port, `exec-form CMD`, and a `HEALTHCHECK`.

Base image choice

The first working version used `python:3.12-slim` (Debian-based). A Trivy scan of that image reported **150 vulnerabilities, 44 of them HIGH**, almost entirely in Debian OS packages unrelated to the application (`util-linux`, `login`, `perl-base`, `bsdutils`, etc.), none of which had an available fix yet. Switching the base image to `python:3.12-alpine` eliminated every OS-level finding (**0 vulnerabilities** on the Alpine layer itself) and reduced the final image size from **~198MB to ~78MB**. `pip`, `setuptools` and `wheel` were also removed after the dependency install, since they are

build-time tools only and were themselves being flagged by the scanner — bringing the image down further to its final size.

Non-root execution

The image creates a dedicated system user/group (appuser/appgroup, uid 100 / gid 101 on Alpine) and switches to it with `USER appuser` before the application ever runs. Confirmed via `docker exec ... id → uid=100(appuser) gid=101(appgroup)`.

Local verification

`docker build`, `docker run`, all routes tested from the host, `docker logs` showing request traffic, `docker inspect` confirming the non-root user and the single exposed port (5000/tcp), and a clean `docker stop/docker rm`. `docker compose up` was also verified, with the container additionally hardened for local testing: `read_only: true` root filesystem (backed by a `tmpfs` mount for `/tmp`), `cap_drop: [ALL]`, and `no-new-privileges:true`. No privileged mode, no Docker socket mount, and no host networking are used anywhere.

3. Security

Vulnerability scan

Tool	Trivy 0.74.0
Image scanned	rebecca16ouatt/msc-de1-flask-app:1.0.0 (final, Alpine-based)
Findings by severity	CRITICAL: 0 · HIGH: 0 · MEDIUM: 0 · LOW: 1
Changes made after scanning	Switched base image Debian→Alpine (removed 150 OS findings); removed pip/setuptools/wheel post-install (removed their own findings)
Remaining findings	CVE-2026-27205 (LOW) — Flask 2.3.3 information disclosure via improper session-data caching. No fix applied because it would require bumping Flask beyond the version pinned by the starter project's <code>requirements.txt</code> , and it is not exploitable here: the application never sets or reads Flask sessions/cookies, so the vulnerable code path is unreachable.

Full output: `security/vulnerability-scan.txt`.

Software Bill of Materials

Generated with Syft 1.51.0 in CycloneDX 1.7 JSON format (`security/sbom.cdx.json`), listing 938 components — every OS package and Python dependency present in the final image.

Other hardening measures

- **Reduced privileges:** `allowPrivilegeEscalation: false` and `capabilities.drop: [ALL]` in Kubernetes; `cap_drop: [ALL]` and `no-new-privileges:true` in Compose.
- **Read-only root filesystem:** enabled in both Compose and Kubernetes, with an `emptyDir/tmpfs` mount for `/tmp` — verified the application still starts, becomes healthy, and serves all routes correctly.
- **Resource awareness:** CPU/memory requests and limits set on the Kubernetes Deployment so the app cannot consume unbounded cluster resources.
- **No secrets in the image:** no credentials appear in the Dockerfile, image layers, or any committed file; the app requires none.

4. Docker Hub

Public repository: <https://hub.docker.com/r/rebecca16ouatt/msc-de1-flask-app>. Two tags were pushed for the final image — `1.0.0` (the exact version used by the Kubernetes Deployment) and `latest` — plus a separate `1.1.0` tag built later purely for the rolling-update demonstration in §6.

Publication was verified end-to-end: the local image was removed (`docker rmi`), pulled back down fresh from Docker Hub (`docker pull rebecca16ouatt/msc-de1-flask-app:1.0.0`), run, and all routes were re-tested successfully against the freshly-pulled image.

5. Kubernetes

Cluster architecture

A local kind cluster (`kind/kind-config.yaml`) with **1 control-plane node and 2 worker nodes**, all reaching Ready status.

Deployed objects (namespace `msc-de1-project`)

Object	Purpose
Namespace	Isolates all project resources under <code>msc-de1-project</code> .
ConfigMap	Externalizes the non-sensitive <code>PORT</code> setting instead of hard-coding it.
Deployment	2 replicas, image <code>rebecca16ouatt/msc-de1-flask-app:1.0.0</code> , explicit container port 5000, readiness & liveness probes on <code>GET /</code> , CPU/memory requests (50m/64Mi) and limits (250m/128Mi), RollingUpdate strategy (<code>maxSurge: 1</code> , <code>maxUnavailable: 0</code>), and a full securityContext: <code>runAsNonRoot: true</code> , <code>runAsUser: 100</code> , <code>runAsGroup: 101</code> (matching the image), <code>allowPrivilegeEscalation: false</code> , <code>capabilities.drop: [ALL]</code> , <code>seccompProfile: RuntimeDefault</code> , <code>readOnlyRootFilesystem: true</code> (with an <code>emptyDir</code> for <code>/tmp</code>).
Service	ClusterIP, selects the Deployment's pods, port 80 → 5000.
NetworkPolicy	Restricts ingress to the app's port to traffic originating from within the same namespace, and egress to DNS only.

Known limitation: kind's default CNI (kindnet) does not enforce NetworkPolicy. The policy correctly documents the intended access model, but is not actively enforced without installing a policy-aware CNI such as Calico or Cilium — out of scope for this project's timeline.

6. Distributed behavior

Replication & service discovery

Both pods reached `Running/Ready` and were scheduled onto the two different worker nodes (`msc-de1-worker` and `msc-de1-worker2`). `kubectl get endpoints` listed both pod IPs behind the Service, and the API was successfully reached through the Service via `kubectl port-forward`.

Self-healing

One running pod was deleted manually (`kubectl delete pod ...`). The Deployment controller detected the discrepancy between desired state (2 replicas) and observed state (1 replica) and created a replacement pod within seconds — no manual intervention required. This is **desired-state reconciliation**: the Deployment/ReplicaSet controller continuously compares the cluster's actual state to the spec and drives convergence, which is fundamentally different from a plain `docker run` container, which stays dead once it exits.

Scaling

`kubectl scale deployment msc-de1-flask-app --replicas=3` brought up a third pod on an available worker node; the Deployment reported 3/3 available shortly after. Scaling back to `--replicas=2` cleanly terminated the extra pod.

Rolling update & rollback

A small, visible version marker was added to the `/` route (`"Hello, Flask! (v1.1.0)"`), the matching unit test was updated, and the image was rebuilt and pushed as `1.1.0`. `kubectl set image deployment/msc-de1-flask-app msc-de1-flask-app=...:1.1.0` triggered a rolling update: `kubectl rollout status` showed pods being replaced one at a time (`maxUnavailable: 0` kept the app continuously available), and `kubectl rollout history` recorded the new revision. The Service immediately began returning the updated text. `kubectl rollout undo` then rolled the Deployment back to the previous revision; both pods returned to image `1.0.0` and the API's response reverted to the original text, confirming a clean rollback.

7. Conclusion

This project required treating the same small Flask application through four different lenses — a bare local process, a Docker container, a Compose service, and a Kubernetes workload — and hardening it consistently at every layer rather than only at the edges. The most concrete lesson was how much a base-image choice affects the security surface: switching from a Debian-slim to an Alpine base collapsed the vulnerability count from 150 (44 HIGH) to a single LOW finding, for less effort than patching individual OS packages one by one. The project also made the practical

difference between a container and a Kubernetes-managed workload tangible rather than theoretical: watching a Deployment recreate a manually deleted pod, or keep the API available throughout a rolling update because of `maxUnavailable: 0`, demonstrates desired-state reconciliation and controlled rollout far more convincingly than reading about them.

One improvement for a real production deployment: replace Flask's built-in development server (which the app currently uses, and which explicitly warns against production use) with a production WSGI server such as Gunicorn behind the same container and Kubernetes setup, and pair it with an Ingress controller and TLS termination instead of relying on `kubect1 port-forward` for access.